Cross Group Evaluation

Monash University Malaysia

Prepared by:

Sahab Ferdous

Taraaf Tazeez Khalidi

Nicolas Kai Xian Weinbrecht

Wong Yew Yi

Date:

18/10/2023

Code Review

Listing files that have syntax, logical or domain-related issues:

`menu.py:`

1. **Indentation**:
    - Consistent and follows Python conventions.
2. **Clarity**:
    - Comments could be more detailed, especially to explain the overall flow or purpose of specific methods.
3. **Variable/Function Names**:
    - Clear and meaningful.
4. **Performance Optimizations**:
    - The login_menu method's nested while loops might be optimised for clarity and simplicity.
    - User input validation is repetitive, and could be optimised by creating another utility function.
5. **Unreachable Code**:
    - None
6. **Error Handling**:
    - Attempts to handle exceptions with try and catch.
7. **Unnecessary Imports:**
    - Learner, Parent, and Educator classes imported but not used.

`login_system.py:`

1. **Indentation**:
    - Consistent and follows Python conventions.
2. **Clarity**:
    - Code is clear and well documented.
3. **Variable/Function Names**:
    - Clear and meaningful.
4. **Performance Optimizations**:
    - The methods username_available, find_account_type, authenticate_user_login, and return_user all iterate over the self.users list to find a user by username. Depending on the size of this list, these repeated searches could be inefficient. It might be better to use a dictionary for faster lookups.
5. **Unreachable Code**:
    - Did not import the ‘User’ class, resulting in an error on line 9.
6. **Error Handling**:
    - Attempts to handle file not found errors with try and except blocks.
    - Could consider other potential errors, like handling incorrect file formats or other unexpected issues during file reading/writing.
7. **Data Security**:
    - Passwords seem to be stored in plaintext in the users.txt file. It's a significant security risk. Passwords could be hashed before storage.
8. **Data Integrity**:
    - Assumes a specific format for the users.txt file (splitting by ;). If the format of the file changes or is corrupted, it might lead to issues. Data validation should be more robust.

challenge.py:

1. **Indentation**:
    - Consistent and follows Python conventions.
2. **Clarity**:
    - Could benefit from more comments to describe its overall purpose and the details of the methods.
3. **Variable/Function Names**:
    - The names used are meaningful and clear.
4. **Performance Optimizations**:
    - None.
5. **Unreachable Code**:
    - None
6. **Error Handling**:
    - Potential exceptions that might arise during user input. For instance, if an unexpected key is missing from the challenge dictionary, it could raise a KeyError.
7. **Data Integrity**:
    - The code assumes that the challenges provided will be in the expected format (i.e., dictionaries with specific keys). It might benefit from additional validation or error handling to ensure data integrity.

learning_modules.py

1. **Indentation**:
    - Consistent and follows Python conventions.
2. **Clarity**:
    - Could benefit from additional comments to further clarify their purpose, such as study_level.
3. **Variable/Function Names**:
    - The names used are meaningful and clear.
    - Class name Learning_Modules (with an underscore) is unconventional and should follow camelCase.
4. **Performance Optimizations**:
    - None.
5. **Unreachable Code**:
    - The code under if __name__ == "__main__": serves as an example and could be removed as it will not be executed when this module is imported elsewhere.
6. **Error Handling**:
    - Attempts to handle invalid user input with try and except blocks.
7. **Data Integrity**:
    - The code assumes that user input for selecting levels will always be a number. If someone enters a non-numeric value, the code will still work, but further validation should be implemented to ensure other inputs are valid (i.e., no negative numbers).

`progress_tracker.py`

1. **Indentation:**
    - Consistent and follows Python conventions**.**
2. **Clarity:**
    - The code is clear.
3. **Variable/Function Names:**
    - The names are clear and meaningful.
4. **Performance Optimizations:**
    - None.
5. **Unreachable Code:**
    - Commented out code on line 8 can be removed.
6. **Error Handling:**
    - The code does not handle potential edge cases. For instance, if the level argument provided to the update_quiz_score or update_challenge_score methods is out of bounds, an IndexError will be raised.

`quiz.py`

1. **Indentation:**
    - Consistent and follows Python conventions.
2. **Clarity:**
    - The code is clear and structured, but could benefit from comments.
3. **Variable/Function Names:**
    - The names used are clear and meaningful.
4. **Performance Optimizations:**
    - None.
5. **Unreachable Code:**
    - None observed in this file.
6. **Error Handling:**
    - No handling for non-integer inputs for the quiz answer. If the user enters a non-integer, a ValueError will be raised.
7. **Data Integrity:**
    - The code assumes that the quiz questions provided will be in the expected format (i.e., dictionaries with specific keys). Additional validation or error handling to ensure data integrity could be implemented.

`educator.py`

1. **Indentation:**
    a. Consistent and follows Python conventions.
2. **Clarity:**
    a. There is minimal code, therefore no comment.
3. **Variable/Function Names:**
    a. The names used are clear and meaningful.
4. **Performance Optimizations:**
    a. None.

5. **Unreachable Code:**
    a. None observed in this file.

`learner.py`

1. **Indentation:**
    - Consistent and follows Python conventions.
2. **Clarity:**
    - There is minimal code, therefore no comment.
3. **Variable/Function Names:**
    - The names used are clear and meaningful.
4. **Performance Optimizations:**
    - None.
5. **Unreachable Code:**
    - None observed in this file.

`parent.py`

1. **Indentation:**
    - Consistent and follows Python conventions.
2. **Clarity:**
    - There is minimal code, therefore no comment.
3. **Variable/Function Names:**
    - The names used are clear and meaningful.
4. **Performance Optimizations:**
    - None.
5. **Unreachable Code:**
    - None observed in this file.

`user.py`

1. **Indentation:**
    - Consistent and follows Python conventions.
2. **Clarity:**
    - Clear as it is a basic representation of a user in the system.
3. **Variable/Function Names:**
    - The names used are clear and meaningful.
4. **Performance Optimizations:**
    - None.
5. **Unreachable Code:**
    - None observed in this file.

Code - Requirements/Design Alignment

Functional Requirements

- Login System aligns with requirements stated.
- Menu aligns with requirements stated.
- Learning Modules align with requirements stated.
- Challenges and Quizzes align with requirements stated.
- Progress Tracker: REQ 31-32 not satisfied. Can't view modules for more detailed progress and no "Recommended Next Step" found.
- Stakeholders' views align with requirements stated.

Non-Functional Requirements

- Performance Requirements not met. NFR-3 not adhered to.
    - No database(other than .txt file).
- Security requirements not met. NFR 8-11 not adhered to.
    - Password complexity is not checked and any password is accepted in the user_register method.
    - No two-factor authentication employed.
    - User actions not logged by the system.
    - Data breaches are not contained and users are not warned of it.

System Architecture

- System architecture has been executed to a great degree of similarity to the diagram.

Use case diagram

- Format of the Use Case Diagrams do not match the conventional style. It is structured more like a flowchart; however, the diagrams accurately depict use cases for the learner and parent/educator.

Class diagram

- LoginSystem class created under controllers package instead of authentication package.
- User class missing attributes and instance methods. Only username and password attributes found.
- account_type attribute not in user class but in children classes instead.
- Menu class mismatch attributes and methods. Missing options attribute
- Level class not shown in diagram.
- Learning_Modules class mismatch attributes and methods.
- Quiz and Challenge class mismatch attributes and methods.
- Progress tracker class mismatch attributes and methods, missing getRecommendedNextStep method.

Sequence diagram

- Parent Class mismatch with the flow in Setting Parent Control sequence.
    - Missing the flow to search for the list of learners under specific parents and change parental controls.
    - Missing the flow to show a list of available parental controls for the learners.

- Educator Class mismatch with the flow in Educator Providing Feedback sequence.
    - Missing the flow to login to educator account type.
    - Missing the flow to search for the learners under the educator.
    - Missing the flow to provide feedback to their learners.